

Worked for 1m 26s

Based on the training tracker, I would run the capstone as a **BMW Connected Mobility Data & AI Platform**. The key design choice is that all 18 participants work under one BMW business theme, but **each participant owns an independent end-to-end use case**, so assessment is individual and there is no risk that one participant's delay blocks another.

Project Management Plan

 Edit



BMW 5-Day Capstone Project Management Plan

1. Project Title

BMW Connected Mobility Data, Analytics & AI Platform

Project Theme

Build production-style BMW solutions using the technologies covered during the course:

Python → PySpark → SQL → AWS → Snowflake → Kafka → FastAPI → QuickSight → AI/LLM/MCP → Terraform → CI/CD → CloudWatch

The capstone will simulate a BMW enterprise environment handling:

- Connected vehicle telemetry
 - Battery and EV information
 - Sales
 - Dealers
 - Service and maintenance
 - Warranty claims
 - Charging sessions
 - Customer feedback
 - Operational events
 - AI-assisted analytics
-

2. Project Duration

5 Working Days

Recommended effort:

- 6–7 development hours per participant per day
 - 30–45 minute morning checkpoint
 - 30 minute evening review
 - Final demonstration on Day 5
-

3. Participants

18 Participants

Each participant receives:

- One independent BMW use case
- One Git repository/project workspace
- Common starter datasets
- Common architecture guidelines
- Standard Definition of Done
- Standard evaluation rubric

For mentoring purposes only, participants can be grouped into six pods.

Pod	Participants	Focus
Pod A	P1–P3	Connected Vehicle Intelligence
Pod B	P4–P6	Service & Maintenance
Pod C	P7–P9	Commercial Analytics
Pod D	P10–P12	Data Engineering & Streaming
Pod E	P13–P15	Data Platform & BI
Pod F	P16–P18	AI & Platform Engineering

Participants remain **individually responsible for their deliverable.**

4. Capstone Objectives

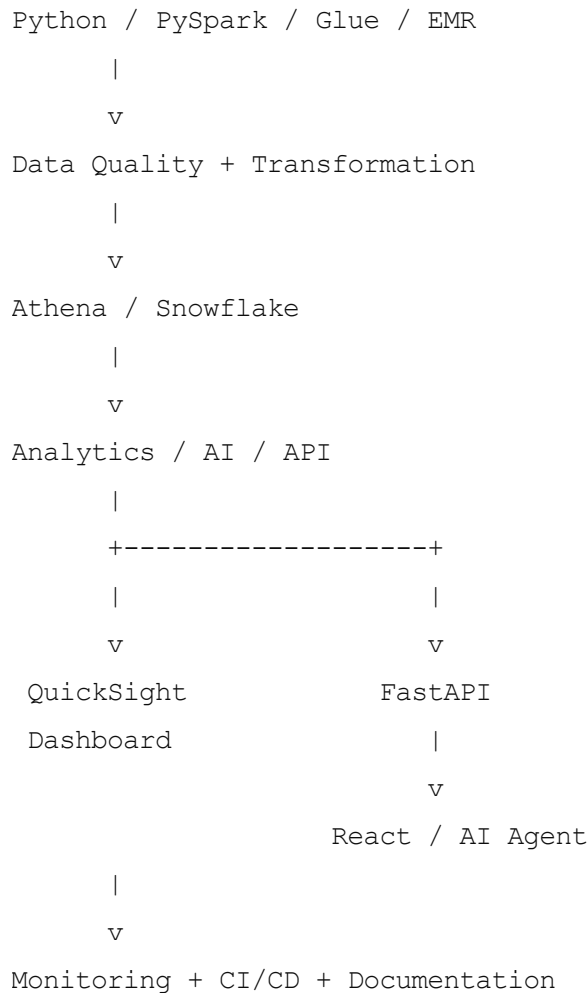
At the end of the five-day project, participants should demonstrate the ability to:

1. Understand a BMW business requirement.
 2. Convert requirements into a technical design.
 3. Design source, staging, processed and analytics data layers.
 4. Implement Python-based processing.
 5. Use PySpark for distributed data transformation where applicable.
 6. Apply SQL for analytical processing.
 7. Use AWS services for data ingestion and processing.
 8. Use Snowflake and/or Athena as an analytical platform.
 9. Implement batch or streaming processing.
 10. Develop APIs using FastAPI where applicable.
 11. Develop analytical dashboards using QuickSight where applicable.
 12. Integrate AI/LLM capabilities where applicable.
 13. Implement automated tests.
 14. Use Git and pull-request practices.
 15. Implement CI/CD.
 16. Provision or describe infrastructure using Terraform.
 17. Implement logging and monitoring with CloudWatch.
 18. Present business insights instead of only demonstrating code.
-

5. Capstone Operating Model

Each participant implements a vertical slice:

```
BMW Data Source
    |
    v
Data Ingestion
    |
    v
S3 / Kafka / Operational Source
    |
    v
```



Not every participant has to use every service.

However, each solution must contain an identifiable:

**Source → Processing → Storage → Business Logic → Output
→ Testing → Monitoring path.**

6. Mandatory Individual Requirements

Every participant must demonstrate the following minimum capabilities.

Common Requirements

Requirement 1 — Python

Use Python for at least one of:

- Data ingestion
- Data validation
- Transformation

- API implementation
- Automation
- AI integration

Requirement 2 — Data Processing

Use either:

- PySpark, or
- Advanced SQL with an appropriate justification.

PySpark projects should demonstrate at least four of:

- filter
- select
- groupBy
- join
- window function
- UDF
- broadcast
- repartition/coalesce
- caching
- null handling

Requirement 3 — AWS

Use:

Amazon S3

plus at least one of:

- Lambda
- SQS
- SNS
- Glue
- EMR
- Athena
- Lake Formation

Requirement 4 — Analytical Data Layer

Use:

- Snowflake, or

- Athena

Advanced projects may use both.

Requirement 5 — Interface

At least one:

- QuickSight dashboard
- FastAPI REST API
- React application
- AI/LLM interface

Requirement 6 — Software Engineering

Must contain:

- Git repository
- README
- requirements.txt / pyproject.toml
- Modular code
- Exception handling
- Logging
- At least five automated tests
- No credentials committed to source control

Requirement 7 — Operations

Must demonstrate:

- CloudWatch logging/metrics OR equivalent application logging
- Error handling
- Failure scenario
- Recovery approach

7. Advanced Technology Distribution

The cohort must collectively cover the complete course.

Technology	Minimum Projects
Python	18

SQL	18
AWS S3	18
PySpark	10+
Snowflake	8+
Athena	6+
Glue	5+
EMR	3+
Lambda	5+
SQS/SNS	3+
Kafka	3+
QuickSight	6+
FastAPI	6+
React	3+
AI/LLM	4+
LangChain/RAG	3+
MCP	2+
Terraform	4+
GitHub Actions	10+
CloudWatch	10+

This approach provides complete course coverage without forcing all 18 participants to build the exact same solution.

8. Project Management Method

Use a simplified Agile/Scrum model.

Sprint

One five-day sprint

Ceremonies

Daily Kick-off — 15 minutes

Each participant answers:

1. What did I complete?
2. What will I complete today?
3. What blocker exists?

Architecture/Technical Clinic — 30 minutes

Trainer resolves:

- AWS permission issues
- Snowflake configuration
- Kafka problems
- Architecture issues
- Data model issues

End-of-Day Review — 30 minutes

Participant demonstrates something executable.

PowerPoint-only progress is not accepted.

9. Project Roles

Trainer

Acts as:

- Project Sponsor
- Product Owner
- Solution Architect
- Technical Reviewer
- Final Evaluator

Participant

Acts as:

- Business Analyst
- Data Engineer
- Backend Developer
- Data Analyst
- Cloud Engineer
- Tester
- DevOps Engineer

for his/her assigned use case.

Peer Reviewer

Each participant receives one peer reviewer.

Reviewer validates:

- Git pull request
- Architecture
- Code quality
- Testing
- Documentation

Reviewer does not write the solution.

10. RACI

Activity	Trainer	Participant	Peer Reviewer
Business requirements	A	R	C
PRD	C	R/A	C
Architecture	A	R	C
Development	C	R/A	-
Data pipeline	C	R/A	-
Testing	C	R/A	C

Code review	C	R	A
Deployment	C	R/A	C
Final demo	A	R	-
Evaluation	R/A	-	C

R = Responsible

A = Accountable

C = Consulted

11. Day-Wise Execution Plan

DAY 1 — Business Understanding, PRD and Architecture

Morning

Kick-off

- Capstone introduction
- Business context
- Use case allocation
- Dataset introduction
- Evaluation criteria

Participant Activities

1. Understand assigned use case.
2. Identify business problem.
3. Identify personas.
4. Define business KPI.
5. Identify datasets.
6. Define functional requirements.
7. Define acceptance criteria.
8. Draw solution architecture.
9. Design data flow.
10. Create Git repository.

Development

Create basic:

- Python project
- Data ingestion
- S3 folders
- Initial schema
- Data validation

Day 1 Deliverables

- Project Charter
- PRD
- Architecture diagram
- Data-flow diagram
- Data dictionary
- Git repository
- Product backlog
- README skeleton

Gate 1

Participant cannot proceed without an approved:

Business problem + Architecture + Acceptance criteria

DAY 2 — Data Engineering

Primary focus:

Python + PySpark + SQL + AWS

Activities

1. Load raw BMW data.
2. Store data in S3.
3. Perform data-quality validation.
4. Handle missing values.
5. Remove duplicates.
6. Standardize data.
7. Apply business transformations.
8. Create calculated fields.
9. Perform joins.

10. Implement aggregation.

11. Store processed data.

Depending on the use case:

- PySpark local
- AWS Glue
- AWS EMR
- Athena
- Snowflake

Suggested S3 Structure

```
bmw-capstone/  
|  
+-- raw/  
|   +-- telemetry/  
|   +-- sales/  
|   +-- dealers/  
|   +-- maintenance/  
|   +-- warranty/  
|  
+-- processed/  
|  
+-- curated/  
|  
+-- rejected/  
|  
+-- logs/
```

Day 2 Deliverables

- Working ETL
- Raw dataset
- Processed dataset
- SQL scripts
- PySpark code
- Data-quality report
- Unit tests

Gate 2

Trainer should be able to see:

DAY 3 — Business Feature Development

Participants implement the major differentiating capability.

Examples:

- Kafka streaming
- Snowflake analytical processing
- FastAPI
- QuickSight
- Machine learning
- RAG
- LangChain
- MCP
- React

Activities

- Business rules
- Analytical queries
- API development
- AI integration
- Dashboard creation
- Event streaming
- Alert logic

Day 3 Deliverables

- Working primary business feature
- API/dashboard/AI/stream
- Updated unit tests
- API documentation where applicable
- Screenshots/results

Gate 3

The business requirement must now be demonstrable.

DAY 4 — Production Engineering

Focus:

Quality + Security + DevOps + Observability

Activities

Testing

- Unit testing
- Negative testing
- Data-quality testing
- API testing

Git

- Branch
- Commit history
- Pull request
- Peer review

CI/CD

GitHub Actions pipeline:

```
Push
|
v
Install Dependencies
|
v
Lint
|
v
Unit Tests
|
v
Build
|
v
Deploy
```

Terraform

Where applicable:

- S3
- IAM
- Lambda
- SQS
- SNS
- Glue
- EMR
- CloudWatch

Monitoring

CloudWatch:

- Logs
- Metrics
- Errors
- Alarms

Day 4 Deliverables

- Tests
- CI/CD pipeline
- Monitoring
- Security checklist
- Deployment instructions
- Terraform or infrastructure specification

Gate 4

Solution should be reproducible and stable enough for demonstration.

DAY 5 — UAT, Documentation and Final Demonstration

Morning

UAT

Test:

- Happy path

- Invalid input
- Missing data
- Duplicate data
- Service failure
- Recovery

Performance

Where applicable:

- Compare SQL execution
- Review Spark partitions
- Review caching
- Review Snowflake query profile
- Review Athena scanned data
- Review streaming latency

Documentation

Complete:

- README
- PRD
- Architecture
- Data model
- API specification
- Testing report
- Deployment steps
- Known limitations
- Future enhancements

Final Presentation

Each participant:

10 minutes — Demonstration

1. Business problem — 1 minute
2. Architecture — 1 minute
3. Data pipeline — 2 minutes
4. Application demonstration — 4 minutes
5. Monitoring/testing — 1 minute
6. Business benefits — 1 minute

12. Git Repository Standard

Recommended structure:

```
bmw-capstone-usecase/  
|  
+-- README.md  
+-- docs/  
|   +-- PRD.md  
|   +-- architecture.png  
|   +-- data_dictionary.md  
|  
+-- data/  
|   +-- sample/  
|  
+-- src/  
|   +-- ingestion/  
|   +-- processing/  
|   +-- api/  
|   +-- ai/  
|   +-- utils/  
|  
+-- sql/  
|  
+-- tests/  
|  
+-- terraform/  
|  
+-- .github/  
|   +-- workflows/  
|  
+-- requirements.txt  
+-- Dockerfile  
+-- .gitignore
```

13. Product Backlog Structure

Each participant creates backlog items such as:

- CAP-01** Setup project repository
 - CAP-02** Create data ingestion
 - CAP-03** Implement validation
 - CAP-04** Implement transformations
 - CAP-05** Create analytical data model
 - CAP-06** Implement business logic
 - CAP-07** Develop API/dashboard
 - CAP-08** Implement testing
 - CAP-09** Configure monitoring
 - CAP-10** Configure CI/CD
 - CAP-11** Documentation
 - CAP-12** UAT and final demo
-

14. Definition of Done

A use case is considered complete only when:

- Business problem is clearly documented.
 - Source data is identified.
 - Architecture is documented.
 - Code runs without manual modification.
 - Data validation exists.
 - Processing logic is implemented.
 - Analytical output is available.
 - Tests execute successfully.
 - Errors are handled.
 - Logs are generated.
 - No passwords/API keys are stored in Git.
 - README contains reproducible instructions.
 - Business KPI is demonstrated.
 - Final demonstration works.
-

15. Evaluation Rubric — 100 Marks

Evaluation Area	Marks
Business understanding & PRD	10

Architecture & data modelling	10
Data engineering implementation	20
Business logic / analytics	20
API / Dashboard / AI capability	10
Code quality & automated testing	10
Cloud / DevOps / CI-CD	10
Security & observability	5
Documentation & final demonstration	5
Total	100

Suggested Rating

- **90–100:** Outstanding
- **80–89:** Excellent
- **70–79:** Very Good
- **60–69:** Good
- **50–59:** Needs Improvement
- **Below 50:** Incomplete

16. Individual Ownership Validation

To prevent one participant receiving credit for another person's work, each participant must demonstrate:

- Own Git commits
- Own architecture
- Own PRD
- Own code
- Own unit tests
- Own deployment
- Live code explanation
- Ability to modify one business rule during evaluation

Trainer may request a live change such as:

Change the high-temperature threshold from 85°C to 90°C and demonstrate the result.

This confirms actual project ownership.

17. Project Risks

Risk	Impact	Mitigation
AWS IAM failure	High	Pre-create sandbox roles/policies
Snowflake access issue	High	Validate accounts before Day 1
Kafka setup problems	Medium	Provide Docker Compose starter
Large data processing time	Medium	Use controlled synthetic data
QuickSight permission issue	Medium	Pre-validate IAM/SPICE
API key exposure	High	Environment variables/secrets
Participant scope too large	High	Freeze MVP after Day 1
Integration issue	Medium	Independent vertical slices
LLM hallucination	Medium	RAG + citations/validation
Time shortage	High	Mandatory MVP + optional stretch goals

18. Project Success Criteria

The capstone succeeds when all 18 participants can independently answer:

“What BMW problem am I solving?”

“Where does my data originate?”

“How is the data processed?”

“Where is it stored?”

“How is the result consumed?”

“How do I know that my solution works?”

“How would this solution operate in production?”

The companion PRD below defines the common BMW product environment and then assigns **18 distinct use cases**. Each one can be independently developed and evaluated.

 Edit



Product Requirements Document

BMW Connected Mobility Data & AI Platform

1. Product Name

BMW Connected Mobility Intelligence Platform — BCMIP

2. Product Vision

Create a unified BMW data, analytics and AI platform capable of processing connected vehicle, customer, commercial, dealer and

service data to provide actionable intelligence to BMW business and engineering teams.

The system demonstrates how modern data engineering, cloud, streaming, BI and generative AI technologies can be combined to solve real BMW-style enterprise problems.

3. Business Problem

BMW generates information from numerous systems:

- Connected vehicles
- EV batteries
- Charging
- Dealerships
- Sales
- Service centres
- Maintenance systems
- Warranty systems
- Customers
- Mobile applications

These datasets frequently arrive at different times and in different formats.

The platform must provide a mechanism to:

1. Ingest the information.
 2. Validate it.
 3. Process it.
 4. Store it.
 5. Analyse it.
 6. Detect important conditions.
 7. Expose insights through APIs.
 8. Visualise information.
 9. Generate AI-assisted insights.
 10. Monitor the complete data pipeline.
-

4. Product Users

Vehicle Operations Manager

Wants visibility into:

- Faults
- Battery condition
- Vehicle temperature
- Vehicle utilisation

Service Manager

Wants:

- Maintenance predictions
- Service workload
- Warranty information

Dealer Manager

Wants:

- Sales trends
- Dealer performance
- Inventory information

Data Analyst

Needs:

- SQL analytics
- Dashboards
- Historical information

Data Engineer

Manages:

- ETL
- Streaming
- Data quality
- Data warehouse

Executive

Needs:

- Business KPIs
- Trends
- Exceptions
- Forecasts

AI Assistant User

Wants to ask:

“Which BMW model had the highest maintenance cost last quarter?”

without manually writing SQL.

5. Core Datasets

Vehicle Master

vehicle_id
vin
model
model_year
fuel_type
region
manufacturing_date

Vehicle Telemetry

event_id
vehicle_id
timestamp
speed
battery_level
temperature
odometer
latitude
longitude
fault_code

Sales

sale_id
vehicle_id

dealer_id
customer_id
sale_date
model
region
price
quantity

Dealer

dealer_id
dealer_name
city
region
capacity
rating

Maintenance

service_id
vehicle_id
dealer_id
service_date
service_type
odometer
parts_cost
labour_cost
failure_code

Warranty

claim_id
vehicle_id
claim_date
component
claim_amount
claim_status

Charging Sessions

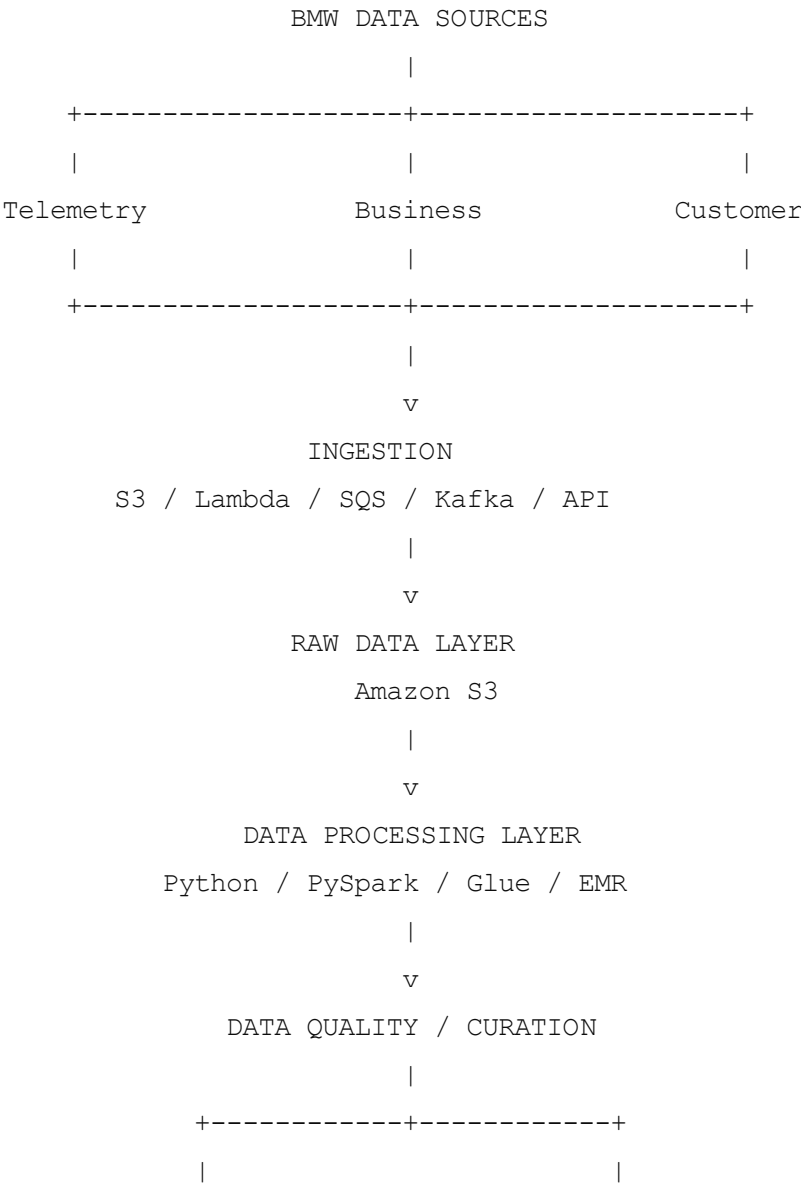
session_id
vehicle_id
station_id
start_time
end_time

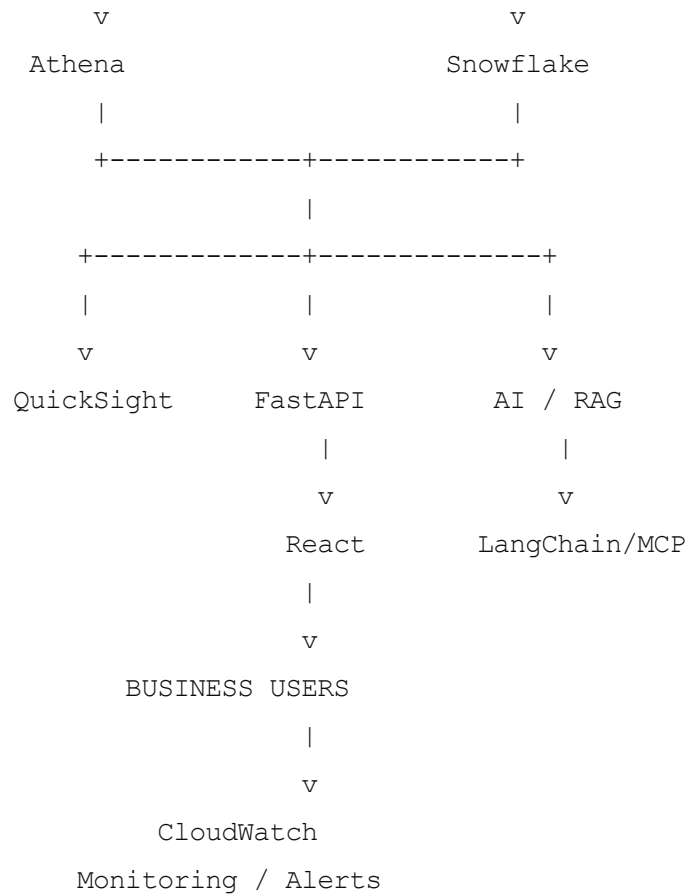
energy_kwh
charging_cost
battery_start
battery_end

Customer Feedback

feedback_id
customer_id
vehicle_id
date
rating
comments
category

6. Logical Architecture





7. Functional Requirements

FR-01 Data Ingestion

The system shall ingest batch datasets into Amazon S3.

FR-02 Streaming

Selected use cases shall ingest real-time events using Kafka.

FR-03 Data Validation

The pipeline shall identify:

- Missing values
- Invalid values
- Duplicate records
- Invalid dates
- Invalid vehicle IDs

Rejected records shall be separated from accepted records.

FR-04 Processing

The platform shall support Python and PySpark transformations.

FR-05 Analytical Query

Processed data shall be queryable using SQL.

FR-06 Data Warehouse

Selected analytical datasets shall be stored in Snowflake.

FR-07 Incremental Processing

Incremental solutions shall avoid reprocessing all historical data.

FR-08 API

Selected applications shall expose data through FastAPI.

FR-09 Dashboard

Selected use cases shall provide business KPIs through QuickSight.

FR-10 AI

Selected applications shall support natural-language interaction with BMW data.

FR-11 Event Alerts

Critical events shall support notification through SNS or equivalent.

FR-12 Testing

Automated tests shall validate major business rules.

FR-13 Monitoring

Application and pipeline failures shall generate logs.

FR-14 CI/CD

Code changes shall trigger automated validation.

FR-15 Infrastructure

Selected cloud resources shall be definable using Terraform.

8. Non-Functional Requirements

Performance

Sample analytical query:

< 5 seconds

for the capstone dataset.

Reliability

Pipeline failures must be logged.

Scalability

Architecture must explain how processing would scale from:

10,000 events → millions of events.

Security

- No embedded passwords.
- IAM least privilege.
- Encryption for cloud storage.
- Secrets supplied using environment variables or secret stores.

Maintainability

- Modular source code.
- Clear function/class names.
- README.
- Documented dependencies.

Testability

At least five automated tests.

Observability

Solutions must expose:

- Success
- Failure
- Processing count

- Error information
-

9. Independent Participant Use Cases

Participant 1 — EV Battery Health Intelligence

Problem

BMW needs to identify vehicles whose batteries are degrading abnormally.

Input

- Telemetry
- Vehicle master
- Charging sessions

Features

Calculate:

- Average battery level
- Charging frequency
- Battery degradation trend
- Battery health category

Technology

Python + PySpark + S3 + Snowflake/Athena + QuickSight.

Output

Battery Health Dashboard.

Acceptance

Vehicle must be classified as:

Healthy / Watch / Critical

based on configurable thresholds.

Participant 2 — Vehicle Overheating & Fault Alert

Problem

BMW operations must identify vehicles reporting dangerously high temperatures.

Input

Real-time telemetry.

Technology

SQS/Kafka → Lambda/Python → SNS → S3.

Business Rule

```
temperature >= configured threshold  
OR  
critical_fault_code exists
```

Output

Alert containing:

- Vehicle
- Temperature
- Fault
- Timestamp

Acceptance

A critical event must generate an alert and create an audit record.

Participant 3 — EV Range & Driving Efficiency Analytics

Problem

Analyse factors affecting EV range.

Input

- Speed
- Battery
- Distance

- Temperature
- Charging

Processing

Use PySpark aggregations and window functions.

Output

- Efficiency by vehicle
- Efficiency by model
- Efficiency by region
- Range trend

Acceptance

Dashboard/API identifies top and bottom efficiency vehicles.

SERVICE & MAINTENANCE

Participant 4 — Predictive Maintenance Risk Score

Problem

Identify vehicles likely to require maintenance.

Inputs

- Telemetry
- Faults
- Odometer
- Maintenance history

Processing

Generate:

`maintenance_risk_score`

Optional

Train a SageMaker model.

Output

Vehicles ranked by maintenance risk.

Acceptance

Return the top 10 highest-risk vehicles with contributing factors.

Participant 5 — Warranty Claims Analytics

Problem

BMW needs to identify high-cost vehicle components.

Input

Warranty + vehicle master.

Technology

PySpark + Snowflake + SQL + QuickSight.

KPIs

- Total warranty cost
- Claims/model
- Claims/component
- Average claim value

Acceptance

Identify the three components generating the highest warranty cost.

Participant 6 — Service Centre Capacity Intelligence

Problem

Identify overloaded and under-utilised service centres.

Input

Maintenance + dealers.

KPIs

- Services/day
- Average repair cost
- Average service interval
- Dealer capacity utilisation

Output

Service operations dashboard.

Acceptance

Flag dealers exceeding configurable capacity thresholds.

COMMERCIAL ANALYTICS

Participant 7 — BMW Sales Trend & Forecast

Problem

Analyse historical BMW sales and generate future sales indicators.

Input

Sales.

Technology

Python + Snowflake + SQL + QuickSight.

KPIs

- Revenue
- Units
- Model growth
- Region growth

Stretch

Forecast next month's sales.

Acceptance

Identify best model and region for the selected period.

Participant 8 — Dealer Performance Scorecard

Problem

Compare dealer performance.

Input

- Dealer
- Sales
- Service
- Customer satisfaction

KPI

Create:

`dealer_performance_score`

using configurable weighted metrics.

Output

QuickSight dashboard.

Acceptance

Rank dealerships and explain the score calculation.

Participant 9 — Dealer Inventory Recommendation

Problem

Recommend which BMW models dealers should stock.

Input

Sales + dealer + inventory history.

Business Logic

Analyse:

- Model demand
- Recent sales
- Regional demand
- Inventory movement

Output

Recommended inventory by dealer/model.

Acceptance

Return:

Dealer

Model

Recommended Quantity

Reason

DATA ENGINEERING & STREAMING

Participant 10 — Enterprise BMW Batch ETL

Problem

Build a reusable pipeline processing multiple BMW datasets.

Input

CSV files in S3.

Pipeline

S3 Raw

|

Glue/EMR PySpark

|

Processed Parquet

|

Glue Catalog

|

Athena

Features

- Schema validation
- Null handling
- Deduplication
- Partitioning
- Parquet conversion

Acceptance

Athena successfully queries curated data.

Participant 11 — Real-Time Connected Vehicle Streaming

Problem

Process live BMW telemetry.

Architecture

```
Telemetry Generator
  |
  v
Kafka
  |
  v
PySpark Structured Streaming
  |
  v
Snowflake / S3
```

Aggregation

Calculate every time window:

- Average speed
- Average battery
- Maximum temperature
- Fault count

Acceptance

New events appear in analytical output without restarting the pipeline.

Participant 12 — BMW Data Quality & Governance Platform

Problem

Data engineers need visibility into data quality.

Checks

- Null percentage
- Duplicate count
- Referential integrity
- Invalid VIN
- Invalid dates
- Out-of-range values

Technology

Python/PySpark + Glue Catalog + Lake Formation + CloudWatch.

Output

Data Quality Score:

0-100

Acceptance

Bad data is quarantined and a quality report is produced.

DATA PLATFORM & BI

Participant 13 — Snowflake Incremental Data Warehouse

Problem

Build a Snowflake data warehouse supporting historical and incremental processing.

Requirements

Create:

- Fact table
- Dimension tables
- Streams
- Tasks
- Stored procedure
- Time Travel example
- Zero-copy clone

Suggested Model

FACT_TELEMETRY

FACT_SALES

DIM_VEHICLE

DIM_DEALER

DIM_DATE

DIM_REGION

Acceptance

New source data is incrementally propagated into analytical tables.

Participant 14 — BMW Executive Analytics Dashboard

Problem

BMW leadership needs a consolidated executive view.

Technology

QuickSight + Athena/Snowflake.

KPIs

- Vehicles monitored
- Sales
- Revenue
- Average battery

- Critical faults
- Warranty cost
- Service volume

Dashboard Features

- Parameters
- Filters
- Drill-down
- KPI cards
- Trend charts
- Regional analysis
- Row-level security

Acceptance

Executive can filter region and drill from overall BMW performance to model level.

Participant 15 — BMW Serverless Data Lake Analytics

Problem

Provide cost-effective serverless analytics.

Architecture

```
S3
|
Glue Crawler
|
Glue Data Catalog
|
Athena
|
QuickSight
```

Requirements

- Raw and curated zones
- Parquet
- Partitioning

- Athena queries
- Glue crawler
- Lake Formation permissions

Acceptance

Demonstrate reduction in scanned data using partitioning/Parquet.

AI & INTELLIGENT APPLICATIONS

Participant 16 — BMW Service Knowledge RAG Assistant

Problem

Technicians need quick answers from BMW service documents.

Architecture

Service Documents

|

v

Text Splitter

|

v

Embeddings

|

v

FAISS / Chroma

|

v

LangChain RAG

|

v

FastAPI

Example

User asks:

What should be checked when an EV reports repeated battery overheating?

Acceptance

Assistant must retrieve relevant documentation before generating the response.

Participant 17 — Natural Language BMW Data Analyst with MCP

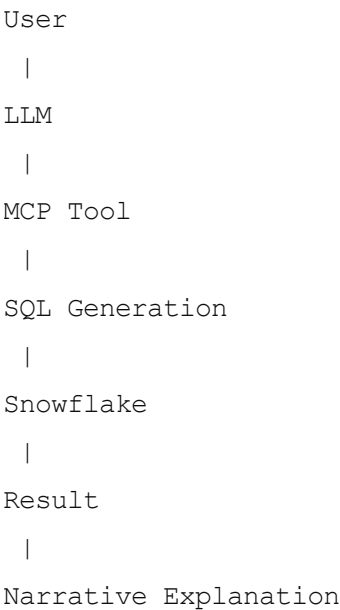
Problem

Business users do not want to manually create SQL.

User Query

Which BMW model had the highest warranty cost in Chennai?

Flow



Requirements

MCP server provides tools such as:

```
get_vehicle_sales()  
get_warranty_cost()  
get_fault_summary()  
get_battery_status()
```

Security

Only read-only approved queries.

Acceptance

User asks a natural-language question and receives data-backed analytical output.

Participant 18 — BMW Data Platform DevOps & Reliability

Problem

BMW data applications require consistent, repeatable and observable deployments.

Terraform

Provision selected resources:

- S3
- IAM
- Lambda
- SQS/SNS
- Glue
- CloudWatch

GitHub Actions

Code Push

|

v

Lint

|

v

Pytest

|

v

Build

|

v

Terraform Plan

|

CloudWatch

Monitor:

- Pipeline failures
- Records processed
- Execution time
- Lambda errors
- Failed jobs

Acceptance

Introduce an intentional failure and demonstrate that:

1. CI/CD detects or blocks an invalid deployment, or
2. CloudWatch captures the operational failure.

10. Course-to-Use-Case Mapping

Course Area	Primary Use Cases
Python	All
OOP	P1, P4, P10, P16
PySpark	P1, P3, P4, P5, P10, P11, P12
SQL	P5, P7, P8, P9, P13, P14, P15, P17
AWS S3	P1–P18 where relevant
Lambda	P2, P10, P18
SQS/SNS	P2, P18
Glue	P10, P12, P15, P18
EMR	P3, P10
Athena	P5, P10, P12, P14, P15

Lake Formation	P12, P15
Snowflake	P1, P5, P7, P8, P11, P13, P14, P17
Kafka	P2, P11
QuickSight	P1, P3, P5, P6, P7, P8, P14, P15
FastAPI	P3, P4, P9, P16, P17
React	P3, P9, P16, P17
SageMaker	P4 optional
LangChain/RAG	P16
MCP	P17
Terraform	P10, P12, P15, P18
GitHub Actions	All
CloudWatch	All / particularly P2, P10, P11, P18

11. Standard Acceptance Checklist

Every participant must answer YES to:

- ☐ Is the BMW problem clearly defined?
- ☐ Is a measurable KPI provided?
- ☐ Is source data available?
- ☐ Is data validation implemented?
- ☐ Is transformed/curated data created?
- ☐ Is SQL or analytical processing demonstrated?
- ☐ Is at least one AWS service used?
- ☐ Is business output provided?
- ☐ Are automated tests included?
- ☐ Is exception handling implemented?
- ☐ Is logging enabled?
- ☐ Are credentials secured?
- ☐ Is the architecture documented?

- ☐ Can another person run the project from the README?
 - ☐ Can the participant explain every major block of the solution?
-

12. MVP vs Stretch Scope

Because the project is only five days, every participant must first complete the **MVP**.

MVP

Data

- Ingestion
- Processing
- Analytical Store
- Business Logic
- Output
- Tests
- Documentation

Stretch Goals

Only after the MVP works:

- Kafka
- React
- SageMaker
- LLM integration
- RAG
- MCP
- Terraform deployment
- Advanced CI/CD
- Advanced observability

This prevents participants from attempting too many technologies and failing to deliver a working solution.

13. Final Capstone Outcome

After five days, the cohort will effectively have created **18 BMW mini-products** demonstrating one shared enterprise architecture.

Together, the projects cover:

Connected Cars + Python + PySpark + Hadoop concepts + SQL + AWS Data Lake + Glue + EMR + Athena + Lambda + SQS/SNS + Snowflake + QuickSight + Kafka + FastAPI + React + AI/LLM + LangChain + RAG + MCP + Terraform + GitHub Actions + CloudWatch.

The project therefore evaluates not only whether participants know individual technologies, but whether they can apply them to a realistic BMW business problem and deliver an independently working solution.

This structure is particularly suitable for the **5-day constraint**: Day 1 freezes scope, Days 2–3 build the core solution, Day 4 makes it production-ready, and Day 5 is reserved for UAT and individual evaluation. It also makes participant scoring much fairer than putting 18 people into one large shared project.